Generating Artwork with Variational Autoencoder

Jen Putnam

May 9, 2022

Abstract

Generative deep learning is a topic of interest to artists and others who want to use AI to generate creative content. This project explores image generation using Variational Autoencoders. VAE models are built from three modules: the encoder transforms input images to a latent representation which has a standard normal distribution, the sampler adds noise to the latent representation, and the decoder reconstructs the latent representation to a unique image similar to the encoded training image. This project attempts to use a VAE to generate colorful images trained from landscape paintings. Experiments include varying model architecture, using data augmentation, using regularization techniques, and varying the size of the learned latent space.

Compared to popular VAE experiments trained on the MNIST dataset, the loss values for this project are high. I think this is due to the large input image size and complexity of generative learning. Latent representations of training images returned by the encoder vary considerably among model architecture experiments. The lowest loss values were achieved when size of the latent space increased, but the generated images got noisy as latent space grew larger.

The best-looking generated images resulted from use of a VAE with a two-dimensional latent space, an encoder architecture with three convolutional layers using filter size 3x3 and using strides value of 2. The number of filters per convolutional layer is 32, 64, and 64, with Batch Normalization layers after each convolutional layer and the final dense layer. This model trained for several thousand epochs on a dataset of 150 mostly homogeneous images with green hills and light skies. I judged this experiment as the most successful because the generated images most closely resembled the training images.

1 Introduction

The purpose of this project is to use deep learning techniques to generate visual artworks. A Variational Autoencoder, or VAE, is a type of neural network capable of generating images. The goal of this project is to build a VAE which will use self-supervised learning to generate artistic landscape images. I will train the model with 1000 images of landscape paintings made by human artists. The VAE will learn to encode the training images into a structured latent space and decode random samples from this latent space into unique artifical images.

I am a formally trained artist, and I am interested in investigating how artifical intelligence learns patterns in artwork. AI models trained on historical artworks often generate images which exhibit a curious combination of style and subject matter. For this project, I am limiting the subject matter to landscapes, although a variety of styles will be represented in the training set. Landscape art has wide commercial value, and there are plenty of available training images.

There is some debate among artists about whether AI-generated art is truly art. I believe it is. The artistic intent is present, and so is the artistic interpretation. Only the creative tools have changed.

2 Dataset

2.1 Data Preprocessing

Since the goal of this project is image generation, not classification, the training samples don't need labels. Image generation using VAEs is a *self-supervised learning* task. The input images themselves

are the targets, and the model will learn these targets on its own. I collected the images used for the training set, by scraping them from Google Images.

The images will be augmented using the Keras ImageDataGenerator class. Specifically, images will be converted to 3D tensors of shape 152 x 152 x 3 before being input to the model for training, and pixel values will be rescaled to values between 0 and 1.

2.2 Example Input Images

Here are some example images from the training dataset:

Figure 1: Examples of Training Images

3 Variational Autoencoder Modeling

A Variational Autoencoder is an extension of a traditional Autoencoder, which uses two modules: encoder and decoder. The encoder reduces dimensionality of the input data. The decoder takes the compressed data from this encoded space (latent space) and reconstructs it back to its original form.

What makes VAEs different from traditional Autoencoders is how they force the encoded latent space into a normal statistical distribution. The resulting latent space has two properties: continuity and completeness. Continuity means that two nearby points in the latent space should generate similar images once decoded. Completeness means that any point sampled from the latent space should decode to a meaningful image, one which resembles the training images. In other words, the latent space should not be sparse, but continuously meaningful along any dimension. This organization of the learned latent space is what makes VAEs capable of image generation. [2]

A Variational Autoencoder is built of two connected convolutional neural networks, the encoder and decoder, with a sampling layer in between. The encoder and decoder are built using the Keras Functional API. The VAE is a subclassed Keras model which combines the encoder and decoder with the custom sampling layer. The modules of a VAE perform these steps:

- The encoder, a convolutional neural network, turns a sample input image into two parameters of the statistical distribution of a two-dimensional latent space: the mean and variance.
- The sampling layer takes a mean and variance as input and adds a small random value to generate a point in the latent space.
- The decoder, another convolutional neural network, takes the point from the latent space and decodes it back to the original image.

3.1 VAE Loss Functions

The VAE learns its parameters by encoding an input image into a latent representation, decoding the latent representation back to an image, and comparing the reconstructed image to the original input image. Two loss functions are used.

Reconstruction Loss measures the difference between reconstructed images and their original inputs, using binary cross-entropy loss. The reconstruction loss informs the weights of the encoder/decoder scheme to achieve better image-generation performance.

Regularization Loss helps organize the latent space by making the distributions returned by the encoder conform to a standard normal distribution centered at 0 (using the Kullback-Leibler divergence)

The VAE model is trained with these loss functions to learn the structure of the latent space it is modeling. [1]

4 Model Architecture Experiments

During this phase of the project, I tested different encoder and decoder architectures and measured them by comparing Total Loss, Regularization Loss, and generated images. Seven model architectures were tested during this phase. Variables include the number of Conv2D layers, number of filters per layer, and strides. The results of this phase are summarized below:

Model Test	Encoder Parameters	Decoder Parameters	Final Total Loss	Final KL Loss	Loss Still Decreasing After 200 Epochs?
Model 1	1,498,132	333,507	14485	5.8	Yes
Model 2	744,500	300,771	14560	6.2	Yes
Model 3	3,033,044	777,603	15520	0.2	No
Model 4	1,535,060	684,547	14480	6.2	Yes
Model 5	3,087,572	832,131	15520	0.1	No
Model 6	1,544,308	379,683	14450	6.1	Yes
Model 7	1,026,420	537,091	15525	0.6	No

Table 1: Results of Phase 3

The table above shows that when the Regularization Loss (KL Loss) is close to zero, Total Loss is higher and stops decreasing quickly. The models which show this pattern are Model 3, Model 5, and Model 7. These models also generated images which are only gray, with no distinct features. When compared with these gray images, the images generated by Model 1, Model 2, Model 4, and Model 6 show more distinct features.

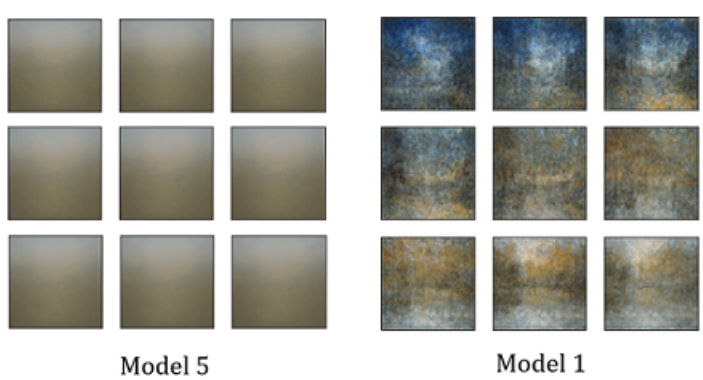

Figure 2: Comparison of Sampled Generated Images

Another observation from this phase was revealed when I tried to map a single input image to its reconstructed output. For every model tested, I followed these steps:

1. I chose a random training image as input. I used the same image for each model test.

2. I sent the image into the encoder, which output the learned latent representation of that image. This latent representation is a mean vector and a variance vector, each with two values.

3. I send the latent representation into the Sampler, which returned a point z in the learned latent space.

4. I sent z into the decoder, and printed the reconstructed output image.

I observed this pattern: for Model 3, Model 5, and Model 7, the encoder returned *smaller* values and the latent space point z was much *closer* to zero. For Model 1, Model 2, Model 4, and Model 6, the encoder returned *larger* latent representation values and the point z was much *further* from zero. Here is the training image I tried to reconstruct for each VAE model:

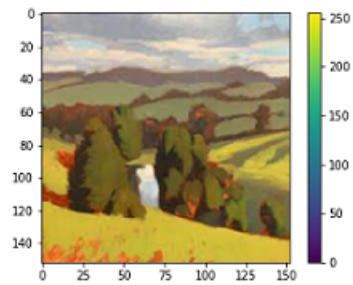

Figure 3: Sampled Image from Training Set

This image was encoded to the following latent representations for each model:

Model	Encoded Mean Vector	Encoded Variance Vector	Latent Point z Returned from Sampler
Model 2	-397.38 -500.85	-504.86 -303.37	-397.38 -500.85
Model 3	0.04 -0.01	-0.96 -1.0	-0.03 -0.78
Model 4	-1518.72 28.63	-2653.34 -2275.81	-1518.72 28.63
Model 5	0.00 0.035	-0.43 -0.41	-0.40 0.15
Model 6	-673.80 -1454.58	-6944.34 -7455.72	-673.80 -1454.58
Model 7	-0.06 -0.02	-0.53 -0.51	0.01 -0.45

Table 2: Latent Representations Learned in Phase 3

I used Model 1, Model 4, and Model 6 for further experiments.

5 Data Augmentation

My main goal during this phase was to investigate high loss values from previous tests. I hypothesized the high loss values may be caused by two things: the large size of the input images and the complexity of the task deep learning for image generation.

I used Model 6 for my experiments in this phase. The first three tests used Keras ImageDataGenerator augmentation techniques, with different values used for each test to vary the amount of augmentation. The final three tests varied the size of the images. The results of this phase are summarized below:

Test	Augmentation Used	Image Size	Final Total Loss	Final KL Loss	Loss Still Decreasing After 200 Epochs?
Test 1	Shift, Zoom, Horizontal Flip	152 x 152 x 3	15,600	0.2	No
Test 2	Shift, Zoom, Horizontal Flip	152 x 152 x 3	14,740	4.6	No
Test 3	Shift, Zoom, Horizontal Flip	152 x 152 x 3	15,670	0.3	No
Test 4	Convert to Grayscale	152 x 152 x 1	14,640	5.8	Yes
Test 5	Reduced Image Size	28 x 28 x 3	526	3.8	No
Test 6	Used MNIST Dataset	28 x 28 x 1	152	2.6	No*

Table 3: Results of Phase 4

*For Test 6, training only lasted 50 epochs, which was sufficient to learn the MNIST dataset.

Model 6 performed worse when ImageDataGenerator techniques were used than when no augmentation was used (see previous phase). The results of Test 4 show that converting my original dataset to grayscale didn't help or hurt the performance of Model 6. Tests 5 and 6 are revealing, though. For these, greatly reduced image size resulted in much lower loss. I think its reasonable to conclude that my high loss values are partly caused by the large input image size.

6 Revised Dataset and Model Architecture, Plus Regularization Techniques

To begin the next phase of the project, I used the three best-performing models from my previous experiments with architecture: Model 1, Model 4, and Model 6. My first set of tests were to increase the final Dense layer from 16 to 32 for each model, and train for 200 epochs on my original dataset. Final loss values were similar for all 3 models. The quality of generated images was also very similar. I chose Model 4 for further experiments with regularization techniques.

None of my previous experiments were improving on the lowest loss of about 14,400. Generated images were still very blurry, and I was unable to map an input image to its output. I asked Dr. Adhikari for advice on working with VAEs. His observations prompted me to make the following changes to my project:

1. I reduced my training dataset to a subset of 150 similar-looking images. I only selected landscapes with green land and light skies. I used no augmentation except resizing images to a uniform 152 x 152.

2. I reduced the size of my encoder and decoder from Model 4 by changing strides to a value of 2 for each Conv2D layer. The goal was for each module to have fewer than 1,000,000 parameters to speed up training.

3. I trained for much longer. I set training to 10,000 epochs with early stopping after 900 epochs without improvement of Total Loss.

Here are some examples of the revised training dataset:

Figure 4: Examples of Training Images: Revised Dataset

6.1 Revised Model 4 Architecture

Here is the encoder architecture used for proceeding experiments. This is Model 4, with the strides value of 2 for all Conv2D layers. It has 795,812 trainable parameters.

Layer Type	Neurons/Filters	Filter Size	Strides	Activation	Padding
Conv2D	32	3	2	ReLU	Same
Conv2D	64	3	2	ReLU	Same
Conv2D	64	3	2	ReLU	Same
Flatten	N/A	N/A	N/A	N/A	N/A
Dense	32	N/A	N/A	ReLU	N/A
Dense (mean)	2	N/A	N/A	N/A	N/A
Dense (variance)	2	N/A	N/A	N/A	N/A

Table 4: New Encoder Architecture Based on Model 4

The last two Dense layers are the vectors which the input image will be encoded to: the mean and variance parameters of the input image's latent representation. Note that downsampling is achieved with strides, not maxpooling. This is because strides are more effective when building a model for which information location is important. This means the model needs to know where things are in an image in order to reconstruct output images successfully. [1]

The mean and variance vectors are then input to a sampling layer which uses these parameters to locate where in the latent space the input image is assumed to have been generated. Then a random point, z, is drawn from close to that location and is output and sent to the decoder. The output z of the sampling layer is derived from this function: z = z_mean + e $\hat{}$(z_log_variance) * epsilon, where epsilon is a random tensor of small values. [1]

Here is the architecture of the decoder, which acts like a mirror of the encoder, outputting images which have equal dimensions to the inputs. The decoder has 162,499 parameters.

Layer Type	Neurons/Filters	Filter Size	Strides	Activation	Padding
Dense	23,104	N/A	N/A	ReLU	N/A
Reshape	19 * 19 * 64	N/A	N/A	N/A	N/A
Conv2DTranspose	64	3	1	ReLU	Same
Conv2DTranspose	64	3	2	ReLU	Same
Conv2DTranspose	32	3	2	ReLU	Same
Conv2D	3	3	1	Sigmoid	Same

Table 5: New Decoder Architecture Based on Model 4

6.2 Regularization Techniques

I did three experiments using the revised Model 4 architecture, while training on the smaller dataset. My first experiment was to add Dropout layers to the model. Dropout layers cancel a random subset of activation values from the previous layer by setting these values to zero. The amount of values dropped is equal to the drop rate, which I set to be 0.3 for all three Dropout layers.

For my second experiment, I added Batch Normalization layers after each Conv2D layer and after the final Dense layer. Batch Normalization layers can be placed between layers of a neural network to normalize the outputs of the preceding layer before passing these outputs as inputs to the next layer. I followed Chollet's advice and placed the ReLU activation *after* the Batch Normalization layers. [1]

For my third experiment, I used the new smaller Model 4 architecture without any regularization techniques. This test was a control, using the same revised, smaller landscape dataset and architecture which I used for the previous two experiments.

I set each experiment to train for 10,000 epochs, but I also used Early Stopping, so the total number of epochs varies. The table below summarizes results of Phase 5.

Regularization Type Used	Final Total Loss	Final KL Loss
Dropout	13,196	6.5
Batch Normalization	13,265	6.5
None (control)	13,176	6.3

Table 6: Results of Tests with Regularization Techniques

The previous three experiments show very similar results in terms of loss and generated images. The changes made to my dataset, model architecture, and length of training have shown improvement. Total loss values have reached lows near 13,000 and the generated images are starting to resemble the training set.

Below are some images generated from a sampled vector of the learned latent space of each experiment.

Figure 5: Using Dropout Rate 0.3

Figure 6: Using Batch Normalization

Figure 7: No Regularization Techniques

I used Batch Normalization for further experiments in Phase 6.

7 Expanding the Latent Space

For Phase 6, I kept what worked best from the previous phases and did three more experiments. Each experiment increased the size of the learned latent space. The encoder module of a VAE performs dimensionality reduction on the orignal inputs, outputting a mean vector and a variance vector which are reduced representations of their input images. These vectors are the learned latent space. When the dimensions of these vectors are increased, the size of the latent space is increased.

My goal for this phase was to investigate if a larger latent space resulted in generated images which preserved more features of the training set. The results of this phase are summarized in the table below.

Latent Space Dimension	Final Total Loss	Final KL Loss
32	12,660	4.8
16	12,715	5.2
8	12,991	5.7

Table 7: Results of Tests Using Larger Latent Space

These results suggest that loss values decrease as the size of the latent space increases. Below are some generated images sampled from the latent space of each experiment.

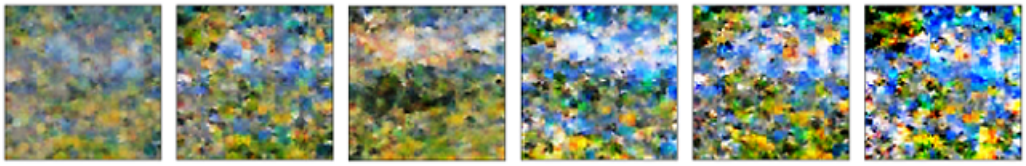

Figure 8: Size of Latent Dimension: 32

Figure 9: Size of Latent Dimension: 16

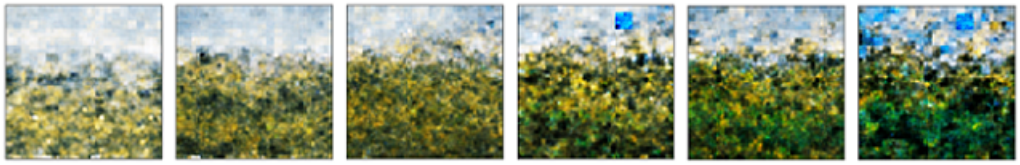

Figure 10: Size of Latent Dimension: 8

While the experiments with latent space dimension resulted in lowest Total Loss values so far, the generated images are noisy and lack high level abstract features of the training set like mountains, trees, and rivers - which were present in images generated during Phase 5.

8 Conclusion

During model architecture experiments of Phase 3, I observed a pattern of lower Regularization Loss corresponding to higher Reconstruction Loss. I still am not sure if this is a result of the number of convolutional layers, or the number of filters per layer, or something else. I would like to know the reason why the training process seems to optimize the Regularization Loss for some models, while optimizing the Reconstruction Loss for others.

When trying to map an input image to a reconstructed output image, the results were not good. For all phases, the same input image was used for this test. For most models tested, latent representations of this image returned by the encoders were very far from zero, in some cases the encoder returned latent representations with values of infinity and negative infinity. I think maybe this means the encoder still has a lot to learn about the latent space of the training data. This could also contribute to high Reconstruction Loss Values.

Data augmentation experiments revealed the size of input images contributed to high Total Loss values. Regularization techniques appear to help the VAE achieve lower loss and better generated images, but only slightly. Increasing the size of the latent space results in lower losses, but noisier generated images which lack abstract features of the training set.

The biggest increase in generated image quality occurred after reducing my training image set to a set of much more homogeneous images and dramatically increasing the training time. Overall, I think the best generated images came from the experiments using the revised Model 4 with Batch Normalization layers, training for about 10,000 epochs on the reduced training set. Below are some images generated by this model (top row), positioned next to similar training set images (bottom row). I selected these generated images by randomly sampling vectors from the learned latent space until I found a few really good ones.

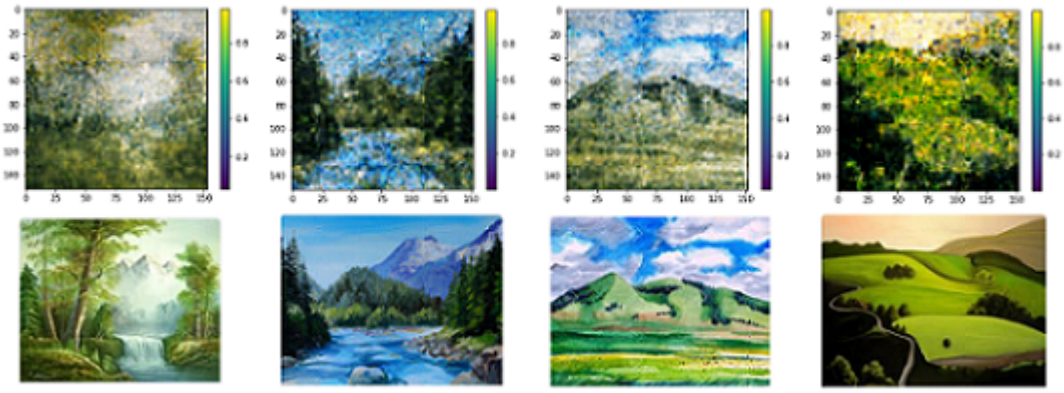

Figure 11: Generated Images and Similar Training Set Images

References

[1] François Chollet. *Deep Learning with Python*. Manning, Nov. 2017. ISBN: 9781617294433.

[2] Joseph Rocca. *Understanding Variational Autoencoders(VAEs)*. Last accessed 13 March 2022. 2019. URL: https://towardsdatascience.com/understanding-variational-autoencoders-vaes-f70510919f73.